

MINI PROJECT 6

ONLINE EXAM PORTAL

Software Architecture & Design Specification

IEEE 1016-style Design Description

Course: Software Engineering

Deliverable: Part-1, Item 3 - Software Architecture & Design Specification

Prepared by: Abhishek Anand

SRN: PES1UG24CS017

Scope: Architecture and detailed design for the Online Exam Portal only.

Document Control

Field	Value
Document Title	Software Architecture & Design Specification - Online Exam Portal
Document ID	OEP-SADS-1.0
Status	Submission Draft
Primary Source	SRS - Online Exam Portal, v1.0
Related Document	Test Plan - Online Exam Portal
Design Standard	IEEE 1016-style software design description

Revision History

Version	Date	Description
1.0	27-Sep-2026	Initial Software Architecture & Design Specification for Part-1 deliverable.

Contents

- ☐ 1. Introduction
- ☐ 2. Design Drivers and Constraints
- ☐ 3. Software Architecture
- ☐ 4. Detailed Design
- ☐ 5. Design-to-Requirement Traceability
- ☐ 6. References

1. Introduction

1.1 Purpose

This document defines the software architecture and detailed design for the Online Exam Portal. It translates the approved SRS into implementable components, interfaces, data structures, interaction sequences, security controls, and error-handling rules. The design is intended to be specific enough to guide implementation and to provide stable interfaces for the accompanying Test Plan.

1.2 System Scope

The Online Exam Portal is a two-role web application for Students and Admins. Admins create MCQ examinations, manage questions, inspect submissions, and release results. Students authenticate, attempt questions in any order, change answers, mark questions for review, submit manually or automatically at timeout, and view scores after results are released.

The design intentionally excludes self-registration, proctoring, plagiarism detection, and non-MCQ question types. These exclusions preserve the mini-project scope defined in the SRS.

1.3 Intended Audience

- ☐ Student developers implementing the browser client and Django backend.
- ☐ Testers preparing API, UI, performance, and security validation.
- ☐ Course evaluators reviewing architecture, traceability, and design completeness.

1.4 Design Conventions

- ☐ Requirement IDs preserve the SRS notation: FR-XX, NFR-XX, SEC-OBJ-XX, and SEC-REQ-XX.
- ☐ REST endpoints use the base path /api/v1. Paths written without that prefix are relative to /api/v1.
- ☐ The term "attempt" means one Student's active or finalized execution of one exam.
- ☐ Attempt states are IN_PROGRESS, SUBMITTED, and AUTO_SUBMITTED.

1.5 Acronyms

Term	Meaning
API	Application Programming Interface
MCQ	Multiple Choice Question
ORM	Object-Relational Mapping
RBAC	Role-Based Access Control
REST	Representational State Transfer
SADS	Software Architecture & Design Specification
SRS	Software Requirements Specification
UI	User Interface

2. Design Drivers and Constraints

2.1 Architectural Goals

- ☐ Keep the system simple enough for a single-semester mini-project while separating presentation, business logic, and data access.
 - ☐ Make exam-attempt state server-authoritative so navigation, saving, submission, grading, and release behavior remain deterministic.
 - ☐ Provide testable REST interfaces aligned with the Test Plan and SRS.
 - ☐ Enforce authentication, role restrictions, Student ownership, transport protection, and post-submission integrity.
- Meet the demonstration-scale performance target of 50 concurrent Students and preserve answers through immediate server-side auto-save.

2.2 Functional Drivers

Requirement Group	Design Driver
FR-01 - FR-02	Separate Student/Admin authentication and role-aware landing behavior.
FR-03 - FR-05	Admin exam/question management and availability-window enforcement.
FR-06 - FR-10	Unrestricted question navigation, answer changes, independent review flag, and five-state navigator grid.
FR-11 - FR-13	Confirmed finalization, timeout finalization, atomic grading, and attempt locking.
FR-14 - FR-17	Admin-controlled result release, Student result view, and Admin submission list.

2.3 Quality and Security Drivers

Requirement	Architectural Response
NFR-01 / NFR-06	Small JSON payloads, stateless request handling, ORM query discipline, and simple relational schema for 50-user demonstration load.
NFR-02	Every answer/review change is persisted through a PATCH request immediately after user action.
NFR-03	Single consistent exam screen with direct navigation and visible five-state status grid.
NFR-05	Explicit layered architecture: presentation, service/business logic, and data access.
NFR-07 SEC-	Browser-based responsive UI with no browser-specific plugin dependencies.
REQ-01..04	Authentication, salted one-way password hashing, RBAC/ownership checks, immutable finalized attempts, and HTTPS-only deployment.

2.4 Constraints and Assumptions

- ☐ Backend technology: Python3.x and Django on a standard Linux server.
- ☐ Persistence: relational database; PostgreSQL is preferred for deployment and SQLite is acceptable for local demonstration.
- ☐ Question type: exactly four options with one correct answer.
- ☐ Accounts are pre-provisioned; account creation and password-reset flows are outside this design.
- ☐ Students are assumed to have stable internet access during the exam window.
- ☐ The mini-project uses a single application deployment; high-availability clustering is not required.

3. Software Architecture

3.1 Architecture Overview

The system uses a layered client-server architecture. Student and Admin browser clients form the presentation layer. Django controllers expose REST endpoints and delegate rules to service components. The service layer contains authentication, exam management, attempt/navigation, submission/grading, and result-release logic. Data access is isolated behind Django ORM repositories over a relational database.

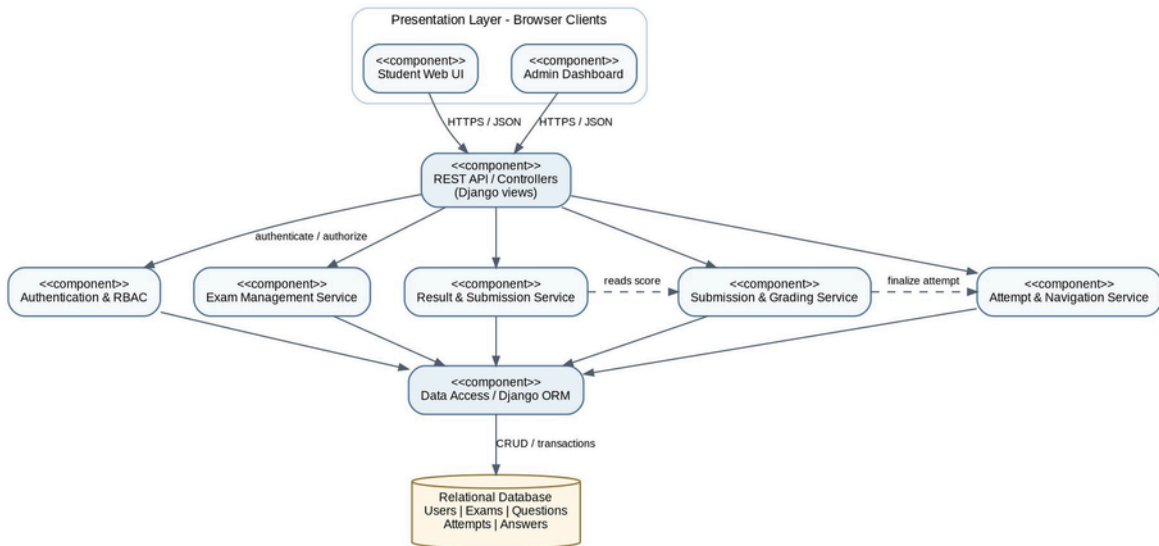


Figure 1. UML-style component diagram for the Online Exam Portal.

3.2 Architectural Pattern

Primary pattern: Layered Architecture combined with RESTful Client-Server interaction. This choice directly supports NFR-05 by separating presentation, business logic, and persistence concerns. Controllers remain thin; business rules are placed in services; database access occurs through the ORM/data-access layer.

Layer	Responsibilities	Representative Components
Presentation	Render Student/Admin screens, manage local navigation state, show timers/status, invoke REST APIs.	Student Web UI, Admin Dashboard
API / Controller	Deserialize requests, validate basic input, establish authenticated principal, map service results to HTTP responses.	Auth, Exam, Attempt, Result controllers
Business / Service	Enforce exam availability, ownership, attempt state, review state, submission, grading, result-release rules.	AuthService, ExamService, AttemptService, GradingService, ResultService
Data Access	Perform ORM queries, transactions, uniqueness checks, and persistence.	Django ORM / repository functions
Data Store	Persist users, exams, questions, attempts, answers, scores and release state.	Relational database

3.3 Component Responsibilities

Component	Key Responsibilities	Requirements
Student Web UI	Question display; direct grid navigation; answer/review controls; countdown; submit confirmation; result page.	FR-05..FR-12, FR-16, NFR-03, NFR-07
Admin Dashboard	Create exams; manage questions; view submissions; release results.	FR-03, FR-04, FR-15, FR-17
Authentication & RBAC	Credential validation, user session/token, role checks, Student ownership checks.	FR-01, FR-02, SEC-REQ-01, SEC-REQ-02
Exam Management Service	Create/update exam definitions; enforce title, duration and availability values; manage MCQs.	FR-03, FR-04, FR-05
Attempt & Navigation Service	Start/read attempts; persist answers and review flags; derive five-state navigator status; enforce deadline and ownership.	FR-06..FR-10, NFR-02
Submission & Grading Service	Manual/auto finalization; atomic score computation; finalized-attempt lock.	FR-11..FR-13, SEC-REQ-03
Result & Submission Service	Hide/reveal scores based on release state; provide Admin attempt list.	FR-14..FR-17
ORM / Database	Transactional persistence, uniqueness and referential integrity.	NFR-02, SEC-OBJ-02

3.4 DeploymentView

The deployment is intentionally minimal: browser clients communicate with one Django application server over HTTPS, and the application server accesses one relational database. This matches the SRS environment and keeps deployment feasible for the course demonstration.

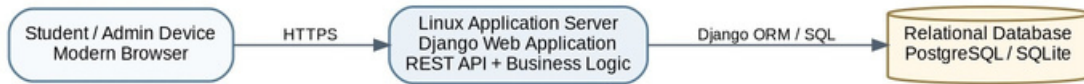


Figure 2. Deployment view.

3.5 Key Architectural Decisions

Decision	Rationale / Consequence
Server-authoritative attempt state	The database is the source of truth for answers, review flags, status, score and finalization. Refreshing the browser therefore reconstructs the same state.
Immediate save through PATCH	Each answer or review change is persisted promptly, supporting the two-second auto-save requirement.
Single submit operation for MANUAL and AUTO modes	Both paths use the same validation and grading code; only the final status differs, reducing inconsistent behavior.
Atomic finalization transaction	Score calculation and attempt locking occur in one database transaction, preventing partial submission and post-submit tampering. Scores may exist immediately after grading but the Student result endpoint denies access until results_released is true.
Admin-controlled release flag	Provides a stable contract for the Test Plan and allows future API evolution without breaking clients.
API version prefix /api/v1	

3.6 Security Architecture

Security is enforced as a sequence of controls rather than a UI-only restriction. All protected requests pass through transport protection, authentication, authorization/ownership checks, and business-state guards before data access.



Figure 3. Security architecture and enforcement path.

Control	Design
Authentication	Every protected endpoint requires an authenticated user. Passwords are stored using Django's configured salted one-way password hasher, never as plaintext.
Role authorization	Admin-only operations verify role = ADMIN. Student operations verify role = STUDENT.
Object ownership	Attempt endpoints verify attempt.student_id equals the authenticated Student ID, preventing cross-account access.
Submission integrity	Only IN_PROGRESS attempts accept answer/review updates. Finalization changes the attempt to SUBMITTED or AUTO_SUBMITTED and all later edits are rejected.
Result confidentiality	The Student result endpoint checks exam.results_released before returning score data.
Transport protection	Production/deployed access is HTTPS-only. HTTP is redirected or refused by deployment configuration.
Error disclosure	Error payloads expose stable codes and user-safe messages, not stack traces, SQL, password hashes, or internal secrets.

4. Detailed Design

4.1 Domain and Data Design

The following logical entities are sufficient for the required behavior. Field names are design-level names; exact Django model syntax may vary without changing the design contract.

Entity	Core Fields	Important Rules
User	id, username, password_hash, role	username unique; role in {STUDENT, ADMIN}; password never stored plaintext.
Exam	id, title, duration_minutes, available_from, available_until, results_released, created_by	duration > 0; available_from < available_until; created_by is Admin.
Question	id, exam_id, text, option_a, option_b, option_c, option_d, correct_option	exactly four options; correct_option in {A,B,C,D}.
Attempt	id, exam_id, student_id, start_time, submitted_time, status, score	one Student/exam attempt if project policy chooses; status controls mutability; score set at finalization.
Answer	id, attempt_id, question_id, selected_option, marked_for_review, visited, saved_at	unique (attempt_id, question_id); selected_option nullable until answered; review flag independent.

4.2 Attempt State and Business Rules

State	Allowed Operations	Transition
IN_PROGRESS	Read questions; navigate; save/change answer; toggle review; manual submit.	Manual submit -> SUBMITTED; deadline auto-submit -> AUTO_SUBMITTED.
SUBMITTED	Read non-sensitive submission status; no answer/review edits; result visible only after release.	Terminal.
AUTO_SUBMITTED	Same restrictions as SUBMITTED.	Terminal.

- ☐ The server computes $\text{deadline} = \text{attempt.start_time} + \text{exam.duration_minutes}$.
- ☐ The browser countdown is informative; the server remains authoritative for whether an attempt is still writable.
- ☐ At the deadline, the browser invokes the same submit endpoint with mode = AUTO. If a write reaches the server after the deadline, the server must not accept a late answer.
- ☐ MarkedforReview never changes grading. Only selected_option is compared with correct_option.

4.3 UML Sequence Diagram 1 - User Login

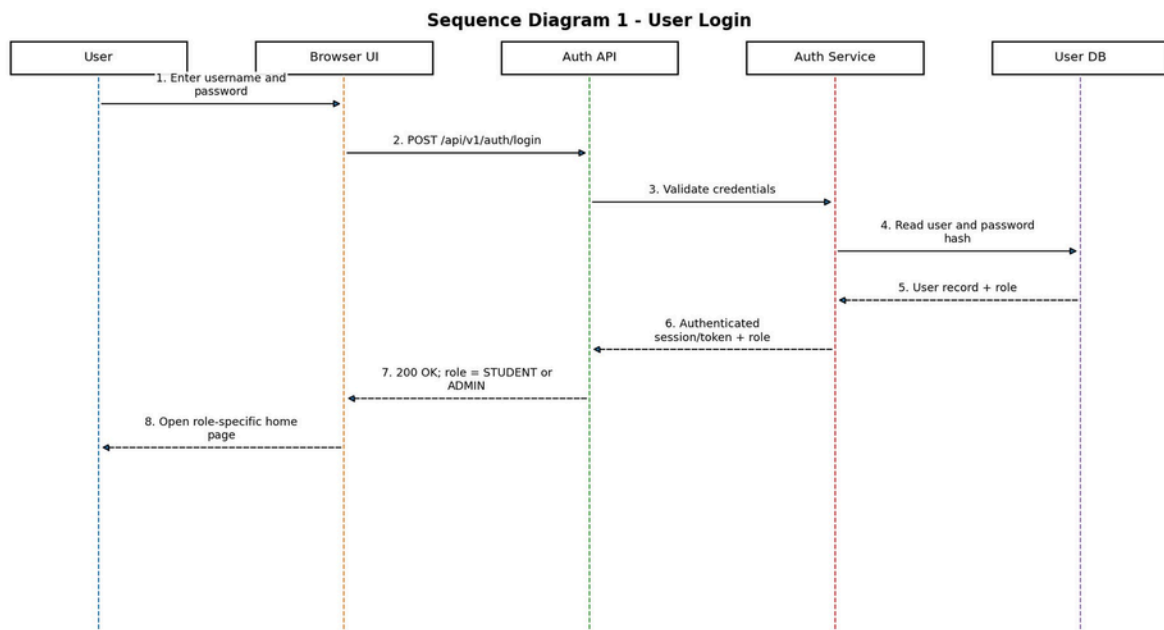


Figure 4. Login interaction for both Student and Admin accounts.

The role returned by authentication determines the permitted UI and API actions. Authentication is a prerequisite for exam and admin functions.

4.4 UML Sequence Diagram 2 - Answer, Review and Auto-save

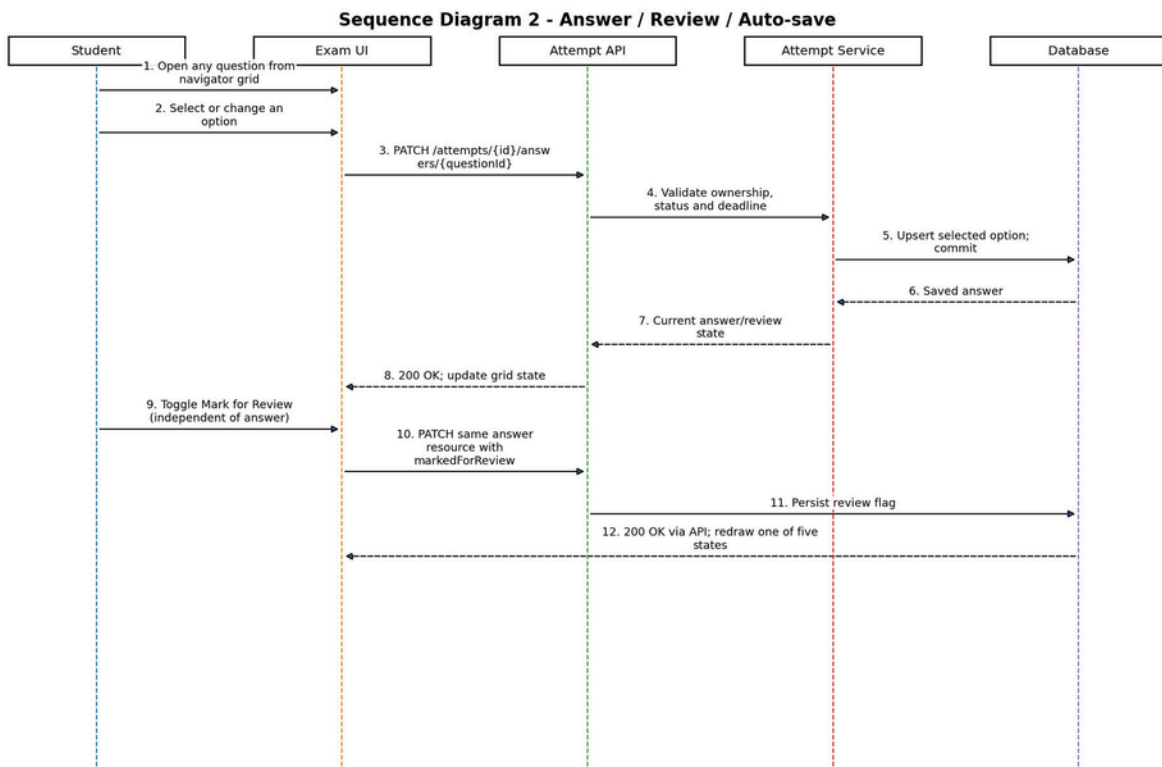


Figure 5. Question interaction, immediate persistence, and navigator-state update.

Answer selection and Mark for Review are persisted independently. The UI derives the navigator state from visited, selected_option, and marked_for_review so that each question displays exactly one of the five SRS-defined states.

4.5 UML Sequence Diagram 3 - Submit, Grade and Release Results

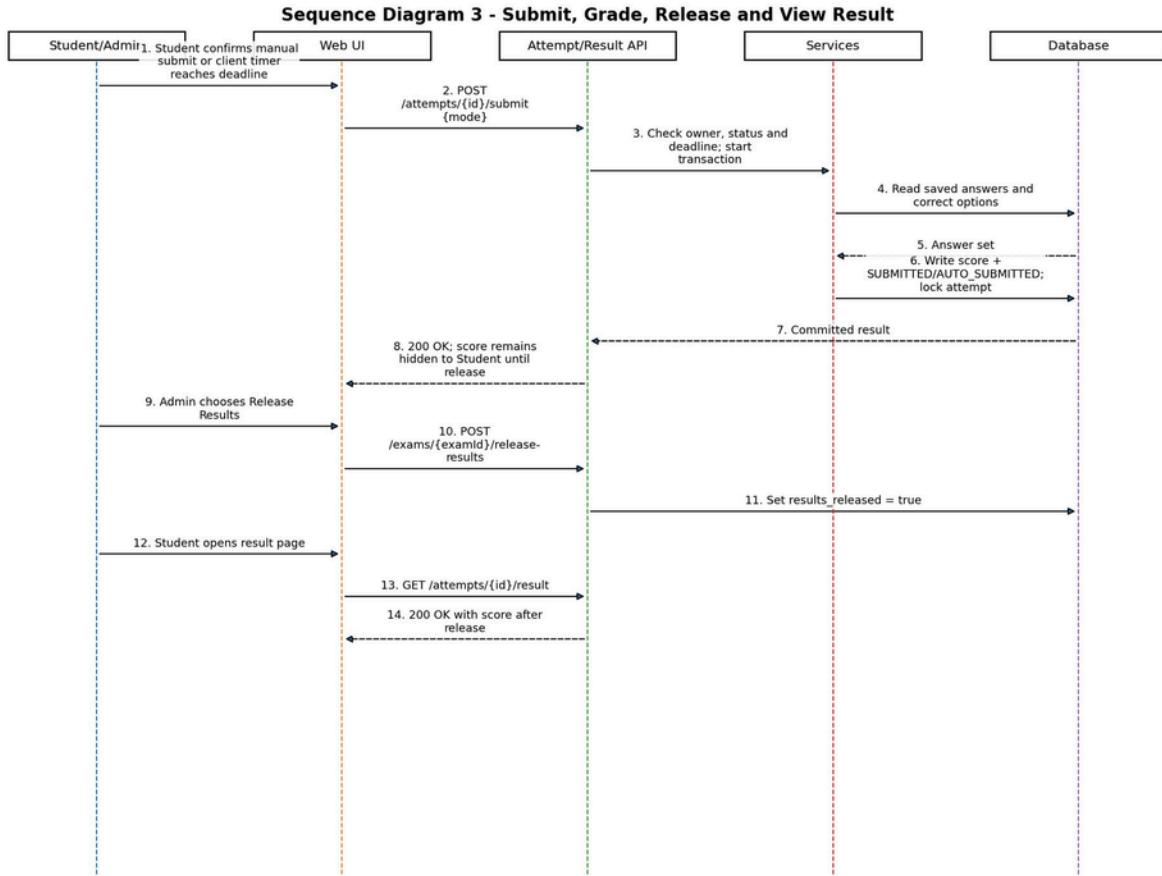


Figure 6. Unified finalization, grading, result release, and Student result access.

Manual and automatic submission share one finalization path. The grading transaction writes the score and final status once. Result release is a separate Admin action, so score computation does not imply Student visibility.

4.6 Question Navigator State Design

The UI must show exactly one state per question. The following precedence removes ambiguity when flags overlap:

Condition	Displayed State
marked_for_review = true AND selected_option is not null	Answered & Marked for Review
marked_for_review = true AND selected_option is null	Marked for Review
selected_option is not null	Answered
visited = true	Not Answered
otherwise	Not Visited

4.7 API Design

4.7.1 API Conventions

- BaseURL: /api/v1. All request and response bodies use JSON unless no body is required.
- Protected endpoints require the authenticated session/token established by login.
- Student ownership and Admin role are validated server-side on every protected operation.
- Create operations return 201 Created; normal reads/updates return 200 OK; failures use the error contract in Section 4.8.

4.7.2 Endpoint Catalogue

Method	Endpoint (relative to /api/v1)	Role	Purpose	Trace
POST	/auth/login	Public	Authenticate Student/Admin and return authenticated context.	FR-01, FR-02
GET	/exams	Student/Admin	List exams visible to the current role.	FR-05
POST	/exams	Admin	Create exam with title, duration and availability window.	FR-03
GET	/exams/{examId}	Student/Admin	Read exam metadata; Student question content only within allowed window. Add one MCQ with exactly	FR-05
POST	/exams/{examId}/questions	Admin	four options and one correct option. Edit an existing MCQ.	FR-04
PUT	/questions/{questionId}	Admin	Delete an MCQ.	FR-04
DELETE	/questions/{questionId}	Admin	Start/retrieve an	FR-04
POST	/exams/{examId}/attempts	Student	IN_PROGRESS exam attempt. Read current attempt, saved answers, review flags and navigator states. Save/change selectedOption and/or markedForReview.	FR-05, FR-06
GET	/attempts/{attemptId}	Owner Student	Finalize MANUAL or AUTO submission and compute score.	FR-06..FR-10
PATCH	/attempts/{attemptId}/answers/{questionId}	Owner Student	Return score only when results are released.	FR-07..FR-10, NFR-02
POST	/attempts/{attemptId}/submit	Owner Student	Set results_released = true.	FR-11..FR-13
GET	/attempts/{attemptId}/result	Owner Student	List attempted Students, submission status and score.	FR-14, FR-16
POST	/exams/{examId}/release-results	Admin		FR-15
GET	/exams/{examId}/submissions	Admin		FR-17

4.7.3 Representative Payloads

Login request / response:

```
POST /api/v1/auth/login
{ "username": "student01", "password": "*****" }

200 OK
{ "role": "STUDENT", "userId": 17, "authenticated": true }
```

Answer/review update:

```
PATCH /api/v1/attempts/42/answers/7
{ "selectedOption": "C", "markedForReview": true }

200 OK
{ "questionId": 7, "selectedOption": "C", "markedForReview": true,
"displayState": "ANSWERED_MARKED_FOR_REVIEW" }
```

Final submission:

```
POST /api/v1/attempts/42/submit
{ "mode": "MANUAL" }

200 OK
{ "attemptId": 42, "status": "SUBMITTED", "scoreComputed": true,
"resultVisible": false }
```

4.8 Error Handling

All API errors use a stable machine-readable code and a user-safe message. Business-rule conflicts are distinguished from authentication, authorization, validation, and not-found errors so the UI and Test Plan can assert exact behavior.

```
{
  "error": {
    "code": "ATTEMPT_ALREADY_FINALIZED",
    "message": "This exam attempt has already been finalized.",
  }
  "requestId": "..."
}
```

HTTP	Error Code	When Used
400	VALIDATION_ERROR	Malformed request, missing required field, invalid duration, option outside A-D, or invalid availability window.
401	AUTHENTICATION_REQUIRED / INVALID_CREDENTIALS	Missing/invalid authentication or failed login.
403	FORBIDDEN	Authenticated user lacks the required role or ownership.
403	RESULTS_NOT_RELEASED	Student requests score before Admin release.
403	EXAM_NOT_AVAILABLE	Student requests protected exam content outside the configured availability window.
404	EXAM_NOT_FOUND /	Requested resource does not exist or is intentionally hidden by access rules.
	ATTEMPT_NOT_FOUND /	
409	QUESTION_NOT_FOUND	Submit is requested after an attempt has already been finalized.
409	DUPLICATE_SUBMISSION	Answer/review update is requested after finalization.
500	INTERNAL_ERROR	Unexpected server failure; details logged server-side, generic message returned to client.
	ATTEMPT_ALREADY_FINALIZED	

- ☐ Database transactions roll back on grading/finalization failure so an attempt is never half-finalized.
- ☐ The UI preserves the current page and shows a concise recoverable error where possible.
- ☐ Authentication/authorization failures never include protected exam, answer, score, or password data in the response.

4.9 Performance, Reliability and Maintainability Design

Concern	Design Response
Performance	Return only exam/attempt data needed by the current screen; keep endpoints stateless; index foreign keys and lookup fields used for exam_id, student_id and attempt_id; avoid N+1 ORM queries.
Auto-save reliability	Persist each change immediately and return the saved state; reloading the attempt reconstructs selected options and review flags from the server.
Concurrency	Use database uniqueness/transaction checks so two late submit requests cannot both finalize the same attempt.
Maintainability	Controllers delegate to services and services to ORM/data-access functions. Question-type logic remains localized, supporting NFR-05.
Portability	Use standards-based HTML/CSS/JavaScript and REST/JSON; no browser plug-ins or platform-specific client features.
Availability	Avoid maintenance during configured exam windows. The single-server mini-project deployment treats NFR-04 as an operational target to be validated during demonstration.

5. Design-to-Requirement Traceability

This matrix shows where each SRS requirement is realized in the architecture/design. It provides the required traceability from requirements to design elements and gives the Test Plan stable components and interfaces to validate.

Requirement	Primary Design Element	Design Location / Mechanism
FR-01	Auth API + Authentication Service	4.7 login endpoint; Fig. 4
FR-02	Auth API + RBAC	4.7 login endpoint; role returned and enforced
FR-03	Admin Dashboard + Exam Service	POST /exams
FR-04	Admin Dashboard + Exam Service	Question create/update/delete endpoints
FR-05	Exam Service + Student UI	Availability checks on exam read/start
FR-06	Student UI + Attempt Service	Direct navigator + attempt read
FR-07	Attempt Service	PATCH answer resource
FR-08	Attempt Service	Independent markedForReview field
FR-09	Student UI + Attempt Service	Section 4.6 five-state precedence
FR-10	Student UI	Navigator links directly to any question
FR-11	Submission Service	POST /attempts/{id}/submit_mode=MANUAL
FR-12	Student UI timer + Submission Service	Same submit endpoint mode=AUTO; server deadline validation
FR-13	Grading Service	Atomic score computation during finalization
FR-14	Result Service	Result endpoint checks results_released
FR-15	Admin Dashboard + Result Service	POST /exams/{id}/release-results
FR-16	Student Result Page + Result API	GET /attempts/{id}/result
FR-17 NFR-01 NFR-02	Admin Dashboard + Result Service	GET /exams/{id}/submissions
NFR-03	API/Service/Data layers	Section 4.9 performance tactics
NFR-04	Attempt Service + ORM	Immediate PATCH persistence; Fig. 5
NFR-05	Student UI	Single exam screen, navigator and explicit action controls
NFR-06	Deployment/operations	Section 4.9 availability response
NFR-07	Layered architecture	Sections 3.1-3.2
SEC-OBJ-01	Stateless Django API + relational DB	Section 4.9 concurrency/performance tactics
SEC-OBJ-02	Browser client	Standards-based responsive UI
SEC-REQ-01	Auth + RBAC + ownership checks	Section 3.6
SEC-REQ-02	Finalization guard + atomic transaction	Sections 3.5; 3.6 and 4.2
SEC-REQ-03	Authentication Service + password hasher	Section 3.6; login endpoint
SEC-REQ-04	RBAC + object ownership	Section 3.6; all protected endpoints
	Attempt state guard	409 errors after finalization
	Deployment configuration	HTTPS-only path in Fig. 3

6. References

- ☐ Software Requirements Specification - Online Exam Portal, Mini Project 6, version supplied for this deliverable.
- ☐ Test Plan - Online Exam Portal, including Task 4 test-case update and security validation.
- ☐ Software Engineering Mini-Project Deliverables Part-1 brief, ETA 28 September 2026.
- ☐ IEEE Std 1016-2009, Systems and software engineering - Design description (used as structural guidance for this SADS).

Submission Checklist

Required Item	Included
Component diagram and description	Yes - Sections 3.1 and 3.3, Figure 1
Architecture pattern	Yes - Section 3.2
Traceability to requirements	Yes - Section 5
Security architecture	Yes - Section 3.6, Figure 3
UML sequence diagrams (at least 2)	Yes - 3 diagrams in Sections 4.3-4.5
API design	Yes - Section 4.7
Error handling	Yes - Section 4.8
Other design sections	Yes - deployment, data design, state rules, quality tactics